You said:

Lab_4_VibeCoding_Student_handout(... PDF

31_galaga-main.zip File

https://github.com/SETAPESU26/31_galaga/tree/main
go through main branch files - readme and game.py and fix the game.py task by task as mentioned in readme, commits should also be made task by task

Worked for 26s

Analyzed

ChatGPT said:

I reviewed the uploaded `31_galaga-main.zip`, including `README.md` and `game.py`, and implemented all four tasks in the order specified in the README. Each task has its own Git commit.

The assignment requires you to push the completed code to your **personal GitHub repository**, not submit a pull request to `SETAPESU26`. I have kept the work separate from the original repository.

1. Download the updated project

31_galaga-task-by-task.zip
Updated game.py, README.md, .gitignore and Git history

Download the updated project ↗

The ZIP includes the `.git` directory so the individual commits are preserved. It does not have a remote configured, preventing an accidental push to the original assignment repository.

2. Changes implemented

Task 1 — Fix cubic Bézier paths

Corrected the second middle term from $3ut$ to $3ut^2$ in both the X and Y calculations. Enemy entry and dive paths now follow the standard cubic Bézier formula.

Commit: `9dee51a` — `fix: correct cubic Bezier middle-term exponents`

Task 2 — Implement `enemy_tint(kind)`

Added wave-dependent enemy colors. Boss, red and blue enemies have distinct colors, and the palette shifts as waves progress. Unknown enemy types fall back to their default colors.

Commit: `2ae71b6` — `feat: tint enemy colors by wave and enemy type`

Task 3 — Implement `on_wave_start(wave)`

Added a centered `WAVE N` banner that displays for one second at the start of each wave. The function also updates the active wave number used by the enemy color system.

Commit: `be693f1` — `feat: show a wave-start banner for one second`

Task 4 — Implement `shield_charges(wave)`

Grants one shield charge on waves 3, 6, 9, 12, and every subsequent third wave. Other waves receive no shield charge. The existing `hit_player()` logic consumes the charge.

Commit: `f71e019` — `feat: grant one shield charge every third wave`

3. Git commit history

The project contains these five commits in chronological order:

Commit	Purpose

`631606e`	Import starter Galaga project
`9dee51a`	Fix Bézier curve formula
`2ae71b6`	Add wave-dependent enemy tinting
`be693f1`	Add wave-start banner
`f71e019`	Add shield charges every third wave

The four implementation commits correspond individually to Tasks 1–4. The initial commit records the original starter project.

4. Validation performed

- Python syntax compilation: passed.
- Bézier start point, end point and midpoint checks: passed.
- Enemy tint changes between waves: passed.
- Distinct colors for the three enemy types: passed.
- Wave banner state and timer initialization: passed.
- Shield schedule for waves 1–6: passed.

Limitation: Pygame is not installed in this execution environment, so I could not launch the graphical game and verify the visual behavior interactively. The logic checks used a lightweight Pygame substitute; they do not replace a full gameplay test.

5. Push the commits to your personal GitHub repository

The GitHub remote could not be accessed from this execution environment, so the commits are prepared locally but have **not been pushed to GitHub**.

After downloading and extracting the ZIP:

1. Create an empty repository under your personal GitHub account, without initializing it with a README or other files.
2. Open PowerShell in the extracted `31_galaga-work` directory.
3. Run:

```
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git push -u origin main
```

Replace `YOUR_USERNAME` and `YOUR_REPO` with your personal GitHub username and repository name. GitHub may prompt you to authenticate.

To verify that all task-by-task commits are present:

```
git log --oneline --reverse
git status
```

Finally, install Pygame in your working Python environment and run the game:

```
python -m pip install pygame
python game.py
```

Since your current setup previously encountered Pygame installation problems with Python 3.14, use the Python 3.12 virtual environment you set up for this project if necessary.

For the assignment submission, remember to record the required before-and-after gameplay videos and export the complete AI chat history, as specified in the supplied lab handout.

‹/›

You said:

i dont want the repo with commits already made, give me fixed code for task 1 first and instructions to fix the code and commit.

Worked for 10s

ChatGPT said:

We will work on **Task 1 only**, using your original repository. You will edit the code yourself, test it, and create the commit locally before moving to Task 2.

Task 1 — Fix the entry/dive path bug

The file to modify is `game.py`. The README asks you to inspect the four weighted terms in the `bezier()` function and correct the exponents in the middle terms.

Step 1: Understand the bug

Open `game.py` and locate this function near the beginning of the file.

Original code (buggy):

```
def bezier(p0, p1, p2, p3, t):
    u = 1 - t
    x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * p2[0] + t ** 3 * p3[0]
    y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * p2[1] + t ** 3 * p3[1]
    return pygame.Vector2(x, y)
```

The problem is in the third weighted term of both coordinates. It uses `3 * u * t` instead of `3 * u * t ** 2`.

The standard cubic Bézier formula is:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$$

Here, P_0 and P_3 are the endpoints, while P_1 and P_2 control the curve's shape.

Step 2: Replace the function

Replace the entire existing `bezier()` function with this corrected version.

```
def bezier(p0, p1, p2, p3, t):
    u = 1 - t
    x = (
        u ** 3 * p0[0]
        + 3 * u ** 2 * t * p1[0]
        + 3 * u * t ** 2 * p2[0]
        + t ** 3 * p3[0]
    )
    y = (
        u ** 3 * p0[1]
        + 3 * u ** 2 * t * p1[1]
        + 3 * u * t ** 2 * p2[1]
        + t ** 3 * p3[1]
    )
    return pygame.Vector2(x, y)
```

Only the missing exponent changes the mathematical behavior; the multiline formatting simply makes the formula easier to inspect.

Step 3: Test the fix

Open PowerShell or the VS Code terminal in your cloned repository directory.

First, compile the Python file:

```
python -m py_compile game.py
```

If the command completes without output, the syntax check passed.

Next, verify that Pygame is installed:

```
python -m pip show pygame
```

If it is installed, launch the game:

```
python game.py
```

Observe the enemies entering the screen and performing dive paths. Their movement should follow smoother, mathematically correct cubic Bézier curves.

You can also test the formula numerically. Create a temporary file named `test_bezier.py` in the repository:

```
import pygame
from game import bezier

p0 = (0, 0)
p1 = (0, 10)
p2 = (10, 10)
p3 = (10, 0)

assert bezier(p0, p1, p2, p3, 0) == pygame.Vector2(*p0)
assert bezier(p0, p1, p2, p3, 1) == pygame.Vector2(*p3)

mid = bezier(p0, p1, p2, p3, 0.5)
assert mid == pygame.Vector2(5, 7.5)

print("Task 1 Bézier tests passed.")
```

Run it:

```
python test_bezier.py
```

Expected output:

```
Task 1 Bézier tests passed.
```

The temporary test file is optional. Do not include it in the commit unless you intend to keep automated tests in your repository.

Step 4: Review the change before committing

Run:

```
git status
git diff -- game.py
```

Confirm that the only intended source-code change is the correction to the two Bézier expressions. If you created the temporary test file, remove it before committing if you do not want it in the repository.

Step 5: Commit Task 1

Stage only `game.py` and create the first task-specific commit:

```
git add game.py
git commit -m "fix: correct cubic Bezier curve formula"
```

Verify the commit:

```
git log -1 --oneline
git status
```

Your commit message should be:

```
fix: correct cubic Bezier curve formula
```

If you have not yet created a personal repository, you can complete and verify this commit locally first. Do not push to `SETAPESU26/31_galaga`.

Stop here after Task 1. Once the tests pass and the commit is created, proceed to Task 2 (`enemy_tint(kind)`) as a separate change and a separate commit. This preserves the task-by-task history required by your lab.

You said:

implement Task 2: Implement `enemy_tint(kind)`

Called once per visible enemy per frame in `draw`, as `color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]`. `kind` is `"boss"`, `"red"`, or `"blue"`. Return an `(r, g, b)` override, or `None` to keep the default from `ENEMY_COLORS`. Idea: recolor enemies by wave number so later waves look visually distinct.

Worked for 22s

ChatGPT said:

Task 2 — Implement `enemy_tint(kind)`

The requirement in `README.md` is to return an RGB color override based on the enemy type and current wave. The existing drawing code already calls:

```
color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]
```

We should implement only Task 2 now and commit it separately. Since `enemy_tint()` receives only `kind`, it needs a module-level wave variable. Task 3's `on_wave_start(wave)` will update that variable when each new wave begins.

Step 1: Add the import and wave variable

At the top of `game.py`, add `colorsys` alongside the existing imports:

```
import colorsys
import math
import random
import pygame
```

Below the existing constants, add:

```
_current_wave = 1
```

This initializes the color palette for Wave 1.

Step 2: Replace the `enemy_tint(kind)` stub

Find the existing function:

```
def enemy_tint(kind):
    """Return an (r, g, b) colour override for an enemy kind, or None for the default."""
    pass
```

Replace it with:

```
def enemy_tint(kind):
    """Return a wave-dependent RGB colour override, or None for the default."""
    base_hues = {
        "boss": 0.32,
```

```
        "red": 0.0,
        "blue": 0.60,
    }

    if kind not in base_hues:
        return None

    hue = (base_hues[kind] + (_current_wave - 1) * 0.085) % 1.0
    rgb = colorsys.hsv_to_rgb(hue, 0.78, 0.95)

    return tuple(round(channel * 255) for channel in rgb)
```

How it works

- **Enemy type:** Each enemy starts with a distinct base hue.
- **Wave number:** The hue shifts by `0.085` per wave, wrapping around after a full color cycle.
- **HSV to RGB:** `colorsys.hsv_to_rgb()` converts the hue, saturation and brightness into RGB components.
- **RGB scaling:** The conversion returns values from `0.0` to `1.0`; multiplying each channel by `255` produces the RGB values Pygame expects.
- **Unknown types:** The function returns `None`, allowing the existing drawing code to use `ENEMY_COLORS`.

The palette uses a fixed saturation of `0.78` and brightness of `0.95` to keep the colors vivid.

Step 3: Test Task 2

Run a syntax check:

```
python -m py_compile game.py
```

If Pygame is installed, run the game:

```
python game.py
```

At this stage, the color function is wave-dependent, but the active wave will remain at Wave 1 until Task 3 updates `_current_wave`. You can test multiple waves directly with this temporary test script.

Create `test_enemy_tint.py`:

```
import game

# Verify all supported enemy types return RGB tuples.
for kind in ("boss", "red", "blue"):
    color = game.enemy_tint(kind)

    assert isinstance(color, tuple)
    assert len(color) == 3
    assert all(0 <= channel <= 255 for channel in color)

# Unknown types should preserve the default-color fallback.
assert game.enemy_tint("unknown") is None

# Verify the palette changes between waves.
game._current_wave = 1
wave_1_color = game.enemy_tint("red")

game._current_wave = 2
wave_2_color = game.enemy_tint("red")

assert wave_1_color != wave_2_color

print("Task 2 enemy tint tests passed.")
```

Run:

```
python test_enemy_tint.py
```

Expected output:

```
Task 2 enemy tint tests passed.
```

The test file is temporary; you do not need to commit it.

Step 4: Review and commit Task 2

Check your changes:

```
git status
git diff -- game.py
```

Ensure the diff contains only the Task 2 additions and changes. If you already committed Task 1, do not amend that commit.

Stage and commit:

```
git add game.py
git commit -m "feat: add wave-dependent enemy tinting"
```

Verify the new commit:

```
git log -2 --oneline
git status
```

Your history should now show two separate implementation commits:

1. `fix: correct cubic Bezier curve formula`
2. `feat: add wave-dependent enemy tinting`

Task 2 is complete once the tests pass and the commit is created. Task 3 will connect wave-start events to `_current_wave`, making the palette change automatically as you progress through the game.

You said:

implement task 3 now

Worked for 25s

ChatGPT said:

Task 3 — Implement `on_wave_start(wave)`

Task 3 requires a **centered `WAVE N` banner displayed for one second** whenever a wave starts. The README specifies that `on_wave_start(wave)` is called automatically by `spawn_wave(wave)`, both when the game starts and when the player clears a wave.

We'll implement this as a separate change, preserving your Task 1 and Task 2 commits.

Step 1: Add banner state variables

In `game.py`, locate the constants at the top of the file. You should already have `_current_wave = 1` from Task 2.

Add these two variables beside it:

```
_current_wave = 1
_wave_banner_wave = 1
_wave_banner_time = 0.0
```

- `_current_wave` controls the wave-dependent enemy colors from Task 2.
- `_wave_banner_wave` stores the wave number shown on screen.

- `_wave_banner_time` tracks how many seconds the banner remains visible.

If `_current_wave` already exists, **do not duplicate it**. Add only the two banner variables.

Step 2: Implement `on_wave_start(wave)`

Find the existing function:

```
def on_wave_start(wave):
    """Called at the start of every wave; add banners, speed-ups, or palette swaps here."""
    pass
```

Replace it with:

```
def on_wave_start(wave):
    """Show a one-second banner and update the active wave palette."""
    global _current_wave, _wave_banner_wave, _wave_banner_time

    _current_wave = max(1, int(wave))
    _wave_banner_wave = _current_wave
    _wave_banner_time = 1.0
```

This updates the active wave for Task 2 and resets the banner duration to one second every time a wave starts.

Step 3: Make the timer count down

Find `Game.update(self, dt, keys)`.

At the beginning of the function, you should see:

```
def update(self, dt, keys):
    if self.state != "play":
        return
    self.time += dt
```

Insert the timer update immediately after the state check:

```
def update(self, dt, keys):
    if self.state != "play":
        return

    global _wave_banner_time
    _wave_banner_time = max(0.0, _wave_banner_time - dt)

    self.time += dt
```

Keep the rest of the existing function unchanged. This uses the frame delta `dt` to count down the banner duration, independent of frame rate.

Step 4: Draw the banner

Find `Game.draw(self, screen, font)`. Near the bottom, you should already have the HUD rendering code:

```
hud = font.render(
    f"Score {self.score}  Lives {self.lives}  Wave {self.wave}  R = reset",
    True,
    (240, 240, 240),
)
screen.blit(hud, (10, 8))
```

Immediately after the HUD rendering and before the existing `if self.state == "lose":` block, insert:

```
if _wave_banner_time > 0:
    banner_font = pygame.font.Font(None, 56)
    banner = banner_font.render(
        f"WAVE {_wave_banner_wave}",
        True,
        (255, 235, 120),
    )
    screen.blit(
        banner,
        banner.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 60)),
    )
```

The banner will appear centered horizontally, slightly above the vertical center, in a bright yellow color. It disappears automatically when the timer reaches zero.

The existing `spawn_wave(wave)` function already calls `on_wave_start(wave)` after creating the enemies, so **you do not need to modify** `spawn_wave()`.

Step 5: Test Task 3

Run these commands in the VS Code terminal from your repository directory.

1. Check Python syntax

```
python -m py_compile game.py
```

No output means compilation succeeded.

2. Launch the game

```
python game.py
```

Verify the following:

- On startup, `WAVE 1` appears near the center of the screen.
- The banner disappears after approximately one second.
- After clearing all enemies, the next wave starts and its banner appears.
- The banner displays the correct wave number.
- Enemy colors still change according to the active wave, as implemented in Task 2.

3. Optional timer test

After the game starts, you can inspect the banner state using a temporary test script:

```
import game

game.on_wave_start(3)

assert game._current_wave == 3
assert game._wave_banner_wave == 3
assert game._wave_banner_time == 1.0

game.on_wave_start(4)

assert game._current_wave == 4
assert game._wave_banner_wave == 4
assert game._wave_banner_time == 1.0

print("Task 3 tests passed.")
```

Save it as `test_wave_start.py` and run:

```
python test_wave_start.py
```

This checks the function's state updates. The graphical test is still needed to verify the banner's appearance and timing.

Step 6: Review and commit Task 3

First, inspect your changes:

```
git status
git diff -- game.py
```

Make sure you have not accidentally changed the Task 1 or Task 2 implementation.

Then create the separate Task 3 commit:

```
git add game.py
git commit -m "feat: show a wave-start banner for one second"
```

Verify your history:

```
git log -3 --oneline
git status
```

Your latest three commits should represent:

1. Task 1 — Fix cubic Bézier curve formula.
2. Task 2 — Add wave-dependent enemy tinting.
3. Task 3 — Show a wave-start banner for one second.

Do not amend earlier commits or push to the original `SETAPESU26` repository.

Stop after Task 3. Once the banner works and the commit is created, Task 4 (`shield_charges(wave)`) can be implemented as the next independent change.

You said:

commited task 3, task 4 now

Worked for 7s

ChatGPT said:

Task 4 — Implement `shield_charges(wave)`

Task 4 is the final feature in your Galaga assignment. We will implement it as a separate change and create a separate Git commit, preserving your commits for Tasks 1–3.

Required behavior: Grant one shield charge on every third wave — waves 3, 6, 9, 12, and so on. All other waves grant zero charges.

Wave	Shield charges
1	0
2	0
3	1
4	0
5	0
6	1
9	1
12	1

Step 1: Implement `shield_charges(wave)`

Open `game.py` and locate the existing `shield_charges(wave)` function. Replace its stub with:

```
def shield_charges(wave):
    """Grant one shield charge every third wave."""
    return 1 if wave > 0 and wave % 3 == 0 else 0
```

This uses the modulo operator (`%`) to check whether the wave number is divisible by three. The function returns an integer, either `1` or `0` .

Do not add a separate shield system if the existing `hit_player()` function already consumes shield charges.

Step 2: Verify the integration

Search `game.py` for these two functions:

- `spawn_wave(wave)` — check that it calls `shield_charges(wave)` and assigns the returned charges to the player's shield state.
- `hit_player()` — check that it consumes a shield charge when the player is hit, rather than losing a life.

If those integrations already exist, leave them unchanged. The task is to implement the missing function, not rewrite working code.

Step 3: Test Task 4

Create a temporary file named `test_shield_charges.py` in your repository:

```
from game import shield_charges

expected = {
    1: 0,
    2: 0,
    3: 1,
```

```
    4: 0,
    5: 0,
    6: 1,
    7: 0,
    8: 0,
    9: 1,
    12: 1,
    15: 1,
}

for wave, expected_charges in expected.items():
    actual = shield_charges(wave)
    assert actual == expected_charges, (
        f"Wave {wave}: expected {expected_charges}, got {actual}"
    )

print("Task 4 shield charge tests passed.")
```

Run the tests in your VS Code terminal:

```
python -m py_compile game.py
python test_shield_charges.py
```

Expected output:

```
Task 4 shield charge tests passed.
```

Then launch the game:

```
python game.py
```

Play until Wave 3 and verify that the shield charge is granted and consumed when the player is hit. Repeat for Wave 6 if practical. The unit test verifies the grant schedule; gameplay verifies the integration.

Step 4: Review and commit Task 4

First, inspect the change:

```
git status
git diff -- game.py
```

Ensure the only intended source-code change is the implementation of `shield_charges(wave)`. Remove the temporary test file if you do not want it included in the repository.

Create the Task 4 commit:

```
git add game.py
git commit -m "feat: grant shield charges every third wave"
```

Verify the commit and working tree:

```
git log -4 --oneline
git status
```

Your latest four commits should now correspond to Tasks 1–4:

1. Fix cubic Bézier curve formula.
2. Add wave-dependent enemy tinting.
3. Show a wave-start banner for one second.
4. Grant shield charges every third wave.

`git status` should show a clean working tree, assuming you removed any temporary test files.

Once Task 4 is committed, all four README implementation tasks are complete. Before submitting, run the full game and capture the required after-change video. Keep the original before-change video and export this chat history for the Lab 4 deliverables.

You said:

Pasted text(20261015-174254).txt
Document

1. `spawn_wave(wave)` — check that it calls `shield_charges(wave)` and assigns the returned charges to the player's shield state.
2. `hit_player()` — check that it consumes a shield charge when the player is hit, rather than losing a life.

these two dont call shield func, fix

ChatGPT said:

I checked your current `game.py`. The two integrations are actually already partially present in the code you shared:

- `spawn_wave(wave)` calls `on_wave_start(wave)`, but it does not assign shield charges itself.
- `Game.reset()` assigns `self.shield = shield_charges(self.wave) or 0`.
- The next-wave logic in `Game.update()` also assigns `self.shield = shield_charges(self.wave) or 0`.
- `Game.hit_player()` already consumes a shield charge before reducing ships or lives.

The main issue is that shield assignment is duplicated in `reset()` and `update()`, rather than being centralized in `spawn_wave(wave)`. Let's fix that while preserving the existing shield-consumption behavior.

Step 1: Update `spawn_wave(wave)`

In `game.py`, replace the existing function with this version:

```
def spawn_wave(wave, game):
    enemies, index = [], 0
    layout = [("boss", 4, 90), ("red", 8, 130), ("red", 8, 170), ("blue", 8, 210)]

    for kind, count, y in layout:
        for i in range(count):
            x = WIDTH / 2 + (i - (count - 1) / 2) * 52
            enemies.append(Enemy(kind, (x, y), index, index % 2 == 0))
            index += 1

    game.shield = shield_charges(wave)
    on_wave_start(wave)
    return enemies
```

The function now assigns the shield charges for the incoming wave directly to the `Game` instance.

Step 2: Update both calls to `spawn_wave`

In `Game.reset()`, change:

```
self.enemies = spawn_wave(self.wave)
self.shield = shield_charges(self.wave) or 0
```

to:

```
self.enemies = spawn_wave(self.wave, self)
```

In `Game.update()`, change the next-wave block from:

```
if not self.enemies:
    self.wave += 1
    self.enemies = spawn_wave(self.wave)
    self.shield = shield_charges(self.wave) or 0
```

to:

```
if not self.enemies:
    self.wave += 1
    self.enemies = spawn_wave(self.wave, self)
```

This removes the duplicate shield assignment.

Step 3: Keep `hit_player()` as follows

Your existing implementation already handles shield consumption correctly. It does **not** need to call `shield_charges()`.

```
def hit_player(self):
    if self.invulnerable > 0:
        return

    if self.shield > 0:
        self.shield -= 1
        self.invulnerable = 1.0
        return

    if self.ships > 1:
        self.ships -= 1
    else:
        self.lives -= 1
        if self.lives <= 0:
```

```
            self.state = "lose"

    self.invulnerable = 2.0
```

Why not call `shield_charges()` **in** `hit_player()`**?** That function determines how many charges a wave grants. Calling it on every hit could replenish a charge repeatedly during a qualifying wave. `hit_player()` should consume the stored charge, not grant another one.

Step 4: Test the integration

Run:

```
python -m py_compile game.py
python test_shield_charges.py
```

If your temporary test file has already been removed, create it again using the test from the previous instructions.

You should also verify that starting a new wave sets `game.shield` correctly. For example, after initializing a game and setting up a wave:

- Wave 2: `game.shield == 0`
- Wave 3: `game.shield == 1`
- Wave 4: `game.shield == 0`
- Wave 6: `game.shield == 1`

Step 5: Commit the integration fix

Since you have not committed Task 4 yet, include these integration changes in your Task 4 commit:

```
git diff -- game.py
git add game.py
git commit -m "feat: grant shield charges every third wave"
git status
```

If you already committed the original Task 4 implementation, make a **new follow-up commit** instead:

```
git add game.py
git commit -m "fix: assign shield charges when spawning waves"
```

This keeps your Task 1–3 history intact and records the shield integration separately.

You said:

instructions to push the completed repo under lab 4 folder in lab repo (didnt create lab repo yet)

Worked for 13s

ChatGPT said:

Push your Galaga project to a new Lab repository

You need to create a personal GitHub repository for your lab submissions, then place the completed Galaga project inside its `Lab-4` folder.

The important requirement is to **preserve your existing Task 1–4 commits**. Do not download a pre-committed ZIP or reinitialize Git inside the Galaga project.

The final repository structure should look like this:

```
your-lab-repo/
  Lab-4/
    31_galaga/
      game.py
      README.md
      .gitignore
      Other project files
```

```
before.mp4
after.mp4
chat-history.pdf
```

The videos and chat-history PDF are included in the structure as placeholders for your assignment deliverables. Add them when ready.

Step 1: Create your Lab repository on GitHub

1. Open GitHub — Create a new repository ↗.
2. Enter a repository name, for example `Lab-Submissions`.
3. Choose **Private** or **Public**, according to your submission requirements.
4. Leave **Add a README file**, `.gitignore`, and license unchecked. Create an empty repository.
5. Copy the HTTPS repository URL, for example:

```
https://github.com/YOUR_USERNAME/Lab-Submissions.git
```

Do not use `SETAPESU26/31_galaga` as the destination.

Step 2: Verify your existing Galaga repository

Open PowerShell in your existing Galaga project directory — the one where you made the four task commits.

Run:

```
git status
git branch --show-current
git log --oneline -5
```

Confirm that:

- Your working tree is clean.

- Your branch is `main`.
- Your Task 1, Task 2, Task 3 and Task 4 commits are present.

If the shield integration fix is still uncommitted, commit it before proceeding.

Step 3: Clone the new Lab repository

Open a separate PowerShell terminal and run:

```
cd "$HOME\Documents"
git clone https://github.com/YOUR_USERNAME/Lab-Submissions.git
cd Lab-Submissions
```

Replace the URL with your actual repository URL.

Step 4: Import Galaga under `Lab-4` while preserving commits

Do not simply copy the project folder containing its `.git` directory into another repository. That would create a nested Git repository rather than include its history in the Lab repository.

Instead, use Git subtree to import the existing Galaga repository and preserve its commit history.

First, create the parent folder:

```
New-Item -ItemType Directory -Path "Lab-4" -Force
```

Now run the following command, replacing the placeholder with the **local path to your existing Galaga repository**:

```
git subtree add --prefix=Lab-4/31_galaga "C:\PATH\TO\YOUR\31_galaga" main
```

For example, if your existing project is in `Documents\31_galaga`:

```
git subtree add --prefix=Lab-4/31_galaga "$HOME\Documents\31_galaga" main
```

This imports the Galaga project into `Lab-4/31_galaga` and retains the imported Git history, including your task-specific commits.

If Git reports that `subtree` is unavailable, stop there rather than deleting the inner `.git` directory or copying over it. You can use an alternative history-preserving Git workflow.

Step 5: Verify the imported history and files

From the root of `Lab-Submissions`, run:

```
git log --oneline --all --graph -15
git log --oneline --all -- Lab-4/31_galaga/game.py
git status
```

Check that `Lab-4/31_galaga/game.py` exists and that the Task 1–4 commits are visible in the imported history.

Step 6: Push the Lab repository

Once the import is successful:

```
git push -u origin main
```

Open your new GitHub repository and verify that `Lab-4/31_galaga/` contains the project files.

Step 7: Add the required lab deliverables

Place your assignment deliverables in `Lab-4/`, alongside the project directory:

- `before.mp4` — 10-second recording before changes.
- `after.mp4` — 10-second recording after all changes.
- `chat-history.pdf` — exported ChatGPT conversation.

Then stage and commit those deliverables separately:

```
git add Lab-4
git commit -m "docs: add Lab 4 submission deliverables"
git push
```

Do not run this final commit until the deliverables have actually been added.

Important: Keep your Galaga task commits separate. The final deliverables commit is an additional submission commit, not a replacement for the individual task commits.

You said:

```
PS C:\Users\vishal\OneDrive\Documents\5th_SEM\SE_LAB\LAB4\SE-labs> git subtree add --prefix=Lab-4/31_galaga
"C:\Users\vishal\OneDrive\Documents\5th_SEM\SE_LAB\LAB4\31_galaga" main
fatal: ambiguous argument 'HEAD': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
fatal: working tree has modifications.  Cannot add.
PS C:\Users\vishal\OneDrive\Documents\5th_SEM\SE_LAB\LAB4\SE-labs>
```

ChatGPT said:

The error indicates that your `SE-labs` repository has **no initial commit yet (`HEAD` does not exist)** and its working tree has modifications. `git subtree add` requires a valid commit in the destination repository.

Let's fix this without losing your existing Galaga task commits.

Step 1: Inspect the destination repository

You are already in:

```
C:\Users\vishal\OneDrive\Documents\5th_SEM\SE_LAB\LAB4\SE-labs
```

Run:

```
git status
git log --oneline -5
git remote -v
```

The `git log` command may fail because there is no initial commit. That's consistent with the error you received.

Step 2: Create the initial commit in `SE-labs`

If `SE-labs` is intended to be a new, empty Lab repository, initialize it with a README:

```
Set-Content README.md "# SE Lab Submissions"
git add README.md
git commit -m "docs: initialize lab submissions repository"
```

If you already have files in `SE-labs` that you need to preserve, **do not overwrite the README or discard those files**. Stage and commit the intended existing files instead.

Step 3: Import Galaga with its task commits

Now run:

```
git subtree add --prefix=Lab-4/31_galaga "C:\Users\vishal\OneDrive\Documents\5th_SEM\SE_LAB\LAB4\31_galaga" main
```

This should import the local Galaga repository under `Lab-4/31_galaga`, preserving its existing Git history.

If the command says the prefix already exists, stop and inspect the directory rather than deleting it.

Step 4: Verify the import

```
git log --oneline --all --graph -15
git status
Get-ChildItem .\Lab-4\31_galaga
```

Confirm that `game.py`, `README.md`, and the other project files are present.

Step 5: Push to GitHub

If `git remote -v` showed your personal GitHub repository as `origin`, run:

```
git push -u origin main
```

If no remote is configured, add your personal repository URL first:

```
git remote add origin https://github.com/YOUR_USERNAME/SE-labs.git
git push -u origin main
```

Replace the URL with your actual repository.

Important: Do not run `git init` again, delete either `.git` directory, or force-push. The steps above preserve your existing project commits while creating a valid destination history.